

THE UNIVERSITY OF DODOMA
COLLEGE OF INFORMATICS AND VIRTUAL EDUCATION



**ATmega32 LDR-Based Automatic
Lighting Control System**

Project Documentation Report

GROUP NUMBER 18

NO.	NAME	REG. NO
1	JOSBERT MEDARD BAINGANA	T23-03-13089
2	GILBERT LAURIAN LUNYUNGU	T23-03-11601
3	ZUWENA CHARLES SANIRA	T23-03-11707
4	IRENE THADEI KISAMBWE	T23-03-10157

1. Problem Statement

Many embedded systems need to respond automatically to environmental light conditions. Examples include street lights, room lighting systems, solar lamps, display brightness controllers, and energy-saving systems. A light-dependent resistor (LDR) changes resistance when light intensity changes. When connected in a voltage-divider circuit, it produces an analog voltage measurable by the ATmega32 ADC.

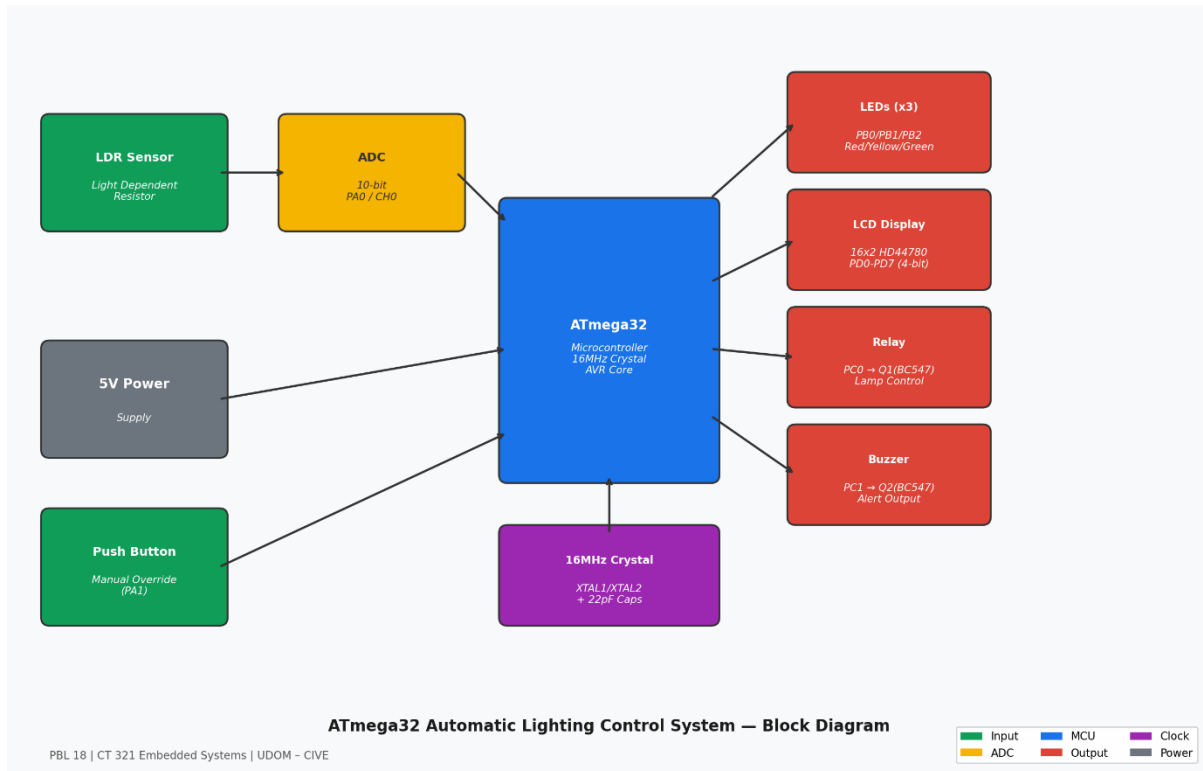
Traditional lighting systems depend on manual switching, leading to energy waste when lights remain ON unnecessarily, safety risks in unlit environments, and inconvenience due to lack of automation. This project addresses these issues by implementing a fully automatic lighting control system that senses ambient light and responds accordingly without human intervention, while still providing a manual override capability.

2. Project Objectives

1. Design and implement an LDR-based automatic lighting control system using the ATmega32 ADC.
2. Develop embedded C firmware implementing ADC reading, 4-state threshold classification, and output control.
3. Control a lamp output (relay/transistor driver) automatically based on measured light level.
4. Provide visual feedback through three LEDs (Green, Yellow, Red) and a 16x2 LCD display.
5. Implement a manual override push button allowing the user to toggle the lamp independently.
6. Include a buzzer alert for very low light conditions and abnormal LDR sensor readings.
7. Design the complete circuit schematic in KiCad and convert it to a PCB layout with Gerber files.
8. Simulate the system in SimulIDE and verify all light-state conditions before hardware implementation.
9. Document all work and upload to a structured GitHub repository with meaningful commit history.

3. System Block Diagram

The diagram below shows the signal flow from the LDR sensor through the ADC to the ATmega32 and its output peripherals.



Signal flow: LDR Sensor → Voltage Divider → ADC (PA0) → ATmega32 → LED / Relay / Buzzer / LCD

4. Main Hardware Components

Component	Qty	Reference	Purpose
ATmega32 Microcontroller	1	U1	Main controller – 16MHz, 10-bit ADC, 4 I/O ports
LDR (Light Dependent Resistor)	1	R5 (LDR03)	Light sensor – resistance varies with light intensity
Fixed Resistor 10kΩ	1	R1	Forms voltage divider with LDR for ADC input on PA0
Fixed Resistor 10kΩ	1	R2	Additional pull-down in voltage divider network
Relay Module / Transistor Driver	1	K1, Q1 (BC547)	Lamp-control output via NPN transistor driver circuit
Green LED	1	D5	Bright light condition indicator – connected to PB2
Yellow LED	1	D6	Medium light condition indicator – connected to PB1
Red LED	1	D4	Low / very low light indicator – connected to PB0
Current-Limiting Resistors 220 ohm	3	R4, R6, R7	LED current protection resistors
Buzzer	1	BZ1	Audio alert for very low light and sensor fault (PC1)
Push Button – Manual Override	1	SW1	Manual lamp toggle input connected to PA1
Push Button – Reset	1	SW2	System reset input connected to MCU RESET pin 9
LCD Module 16x2	1	DS1 (WC1602A)	Displays system light status and ADC state messages
Crystal Oscillator 16MHz	1	Y1	External clock source for ATmega32 operation
Capacitors 22pF	2	C1, C2	Crystal load capacitors on XTAL1 and XTAL2
Capacitors 100nF	2	C3, C4	Power supply decoupling and ADC noise filtering
Electrolytic Capacitor 1uF	1	C6	Reset circuit debounce capacitor
Pull-up Resistor 10kΩ	1	R11	RESET pin pull-up to VCC
Power Supply 5V	1	—	5V regulated DC supply for the entire circuit

5. ATmega32 Pin Mapping Table

The table below shows the complete pin assignment for all ATmega32 I/O pins used in this project.

ATmega32 Pin	Port	Phys. Pin	Direction	Connected To	Function
PA0 / ADC0	PORTA	40	INPUT	LDR Voltage Divider	ADC Channel 0 – light level reading
PA1	PORTA	39	INPUT	Push Button SW1	Manual override toggle (Active LOW, internal pull-up)
PB0	PORTB	1	OUTPUT	Red LED D4 + R7 (220 ohm)	Low light and error indicator
PB1	PORTB	2	OUTPUT	Yellow LED D6 + R6	Medium light indicator
PB2	PORTB	3	OUTPUT	Green LED D5 + R4	Bright light indicator
PC0	PORTC	22	OUTPUT	Q1 Base via R3 (1kΩ)	Relay driver – lamp control output
PC1	PORTC	23	OUTPUT	Q2 Base via R8 (1kΩ)	Buzzer driver – audio alert output
PD0	PORTD	14	OUTPUT	LCD RS Pin 4	LCD register select signal
PD1	PORTD	15	OUTPUT	LCD EN Pin 6	LCD enable strobe signal
PD4	PORTD	18	OUTPUT	LCD D4 Pin 11	LCD 4-bit data bus bit 4
PD5	PORTD	19	OUTPUT	LCD D5 Pin 12	LCD 4-bit data bus bit 5
PD6	PORTD	20	OUTPUT	LCD D6 Pin 13	LCD 4-bit data bus bit 6
PD7	PORTD	21	OUTPUT	LCD D7 Pin 14	LCD 4-bit data bus bit 7
VCC	—	10	POWER	+5V Supply	Main digital supply
GND	—	11	POWER	Ground	Digital ground
AVCC	—	30	POWER	+5V Supply	ADC analog supply
AGND	—	31	POWER	Ground	ADC analog ground
RESET	—	9	INPUT	R11 + SW2 + C6	Active LOW reset with pull-up and debounce
XTAL1	—	13	INPUT	Crystal Y1 + C1 (22pF)	External 16MHz clock input
XTAL2	—	12	OUTPUT	Crystal Y1 + C2 (22pF)	External 16MHz clock output

DDR Configuration: DDRA = 0x00 | DDRB = 0x07 | DDRC = 0x03 | DDRD = 0xF3

LCD R/W pin tied to GND (write-only mode). LCD Vo/contrast pin tied to GND for maximum contrast.

6. LDR Voltage Divider Circuit

The LDR is connected in a voltage divider configuration with a fixed $10\text{k}\Omega$ resistor (R1). The output voltage at the midpoint is fed to PA0 (ADC Channel 0) of the ATmega32.

```
VCC (+5V)
|
R1 (10kohm) <-- Fixed resistor
|
+-----> PA0 / ADC0 (ADC input to ATmega32)
|
LDR (R5) <-- Resistance varies inversely with light intensity
|
GND (0V)
```

Operating Principle

When light intensity is HIGH, LDR resistance is LOW. Most of the supply voltage drops across R1 and the voltage at PA0 is HIGH (close to 5V). The ADC reads a high value and the system enters the BRIGHT condition.

When light intensity is LOW (dark), LDR resistance is HIGH. Most voltage drops across the LDR and the voltage at PA0 is LOW (close to 0V). The ADC reads a low value and the system enters the DARK condition with relay ON and buzzer alert.

7. ADC Threshold Table

The ATmega32 10-bit ADC produces values from 0 to 1023. The firmware classifies light conditions using the following thresholds with AVCC = 5V as the ADC reference voltage.

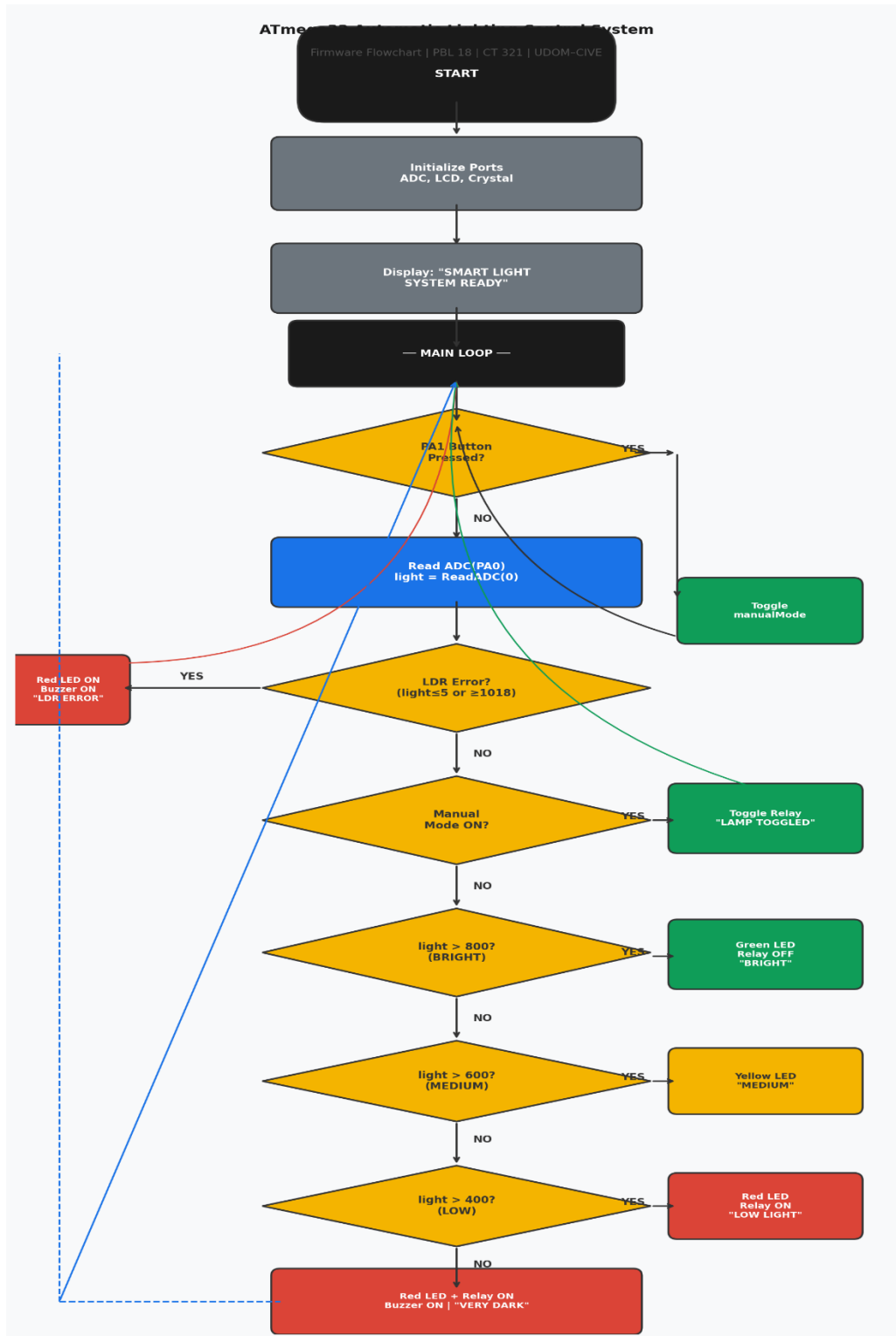
State	ADC Condition	ADC Range	Voltage (approx)	Relay	Buzzer	LED	LCD Message
BRIGHT	light > 800	801 – 1017	3.92V – 4.97V	OFF	OFF	Green PB2	"BRIGHT"
MEDIUM	light > 600	601 – 800	2.94V – 3.91V	OFF	OFF	Yellow PB1	"MEDIUM"
LOW	light > 400	401 – 600	1.96V – 2.93V	ON	OFF	Red PB0	"LOW LIGHT"
DARK	light <= 400	6 – 400	0.03V – 1.95V	ON	ON	Red PB0	"VERY DARK"
ERROR	light <= 5 or >= 1018	0-5 / 1018+	< 0.02V / > 4.98V	OFF	ON	Red PB0	"LDR ERROR"

Thresholds defined in firmware: **BRIGHT_THRESHOLD = 800**,
MEDIUM_THRESHOLD = 600, **LOW_THRESHOLD = 400**.

Values may be adjusted by the instructor based on the LDR voltage-divider connection characteristics.

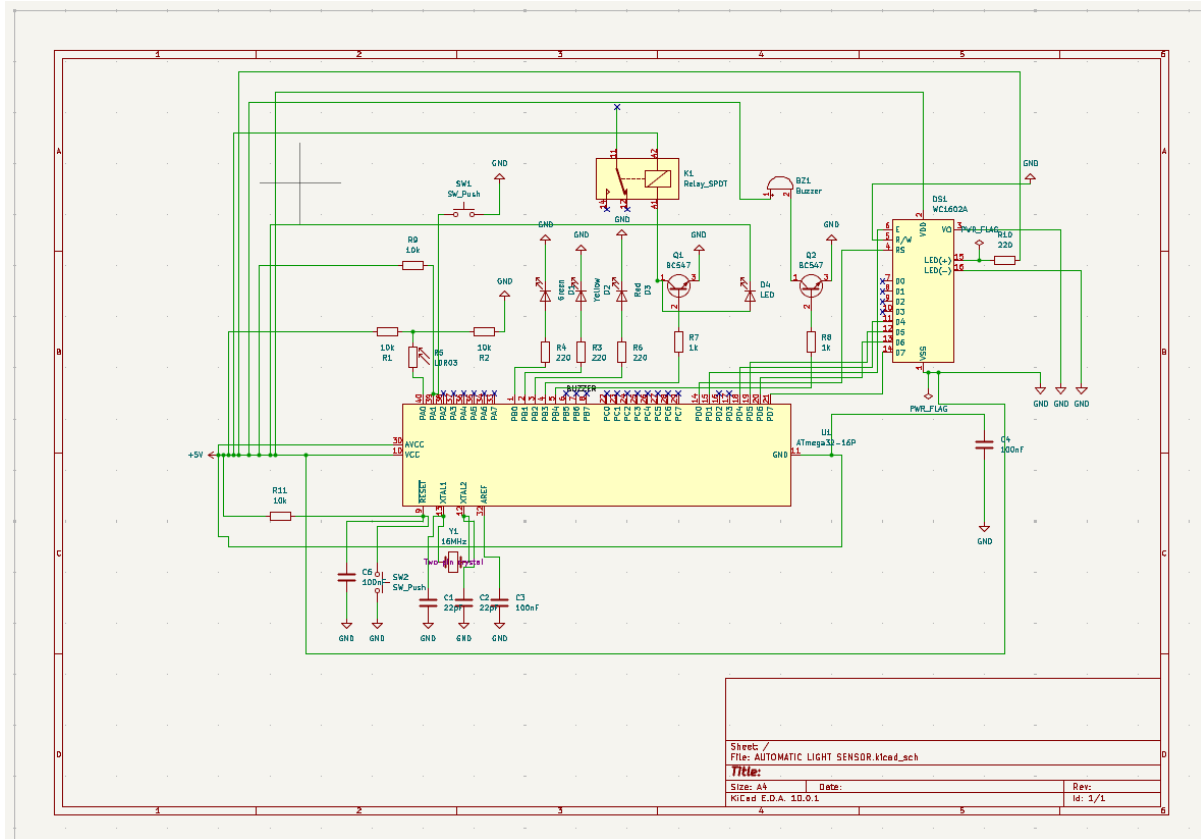
8. Firmware Flowchart

The flowchart below illustrates the firmware decision logic implemented in main.c, showing the complete execution path from initialization through the main control loop.



9. KiCad Schematic Design

The complete circuit schematic was designed in KiCad 10.0. The schematic includes all components, net labels, power connections, and driving circuits as required by the project brief.

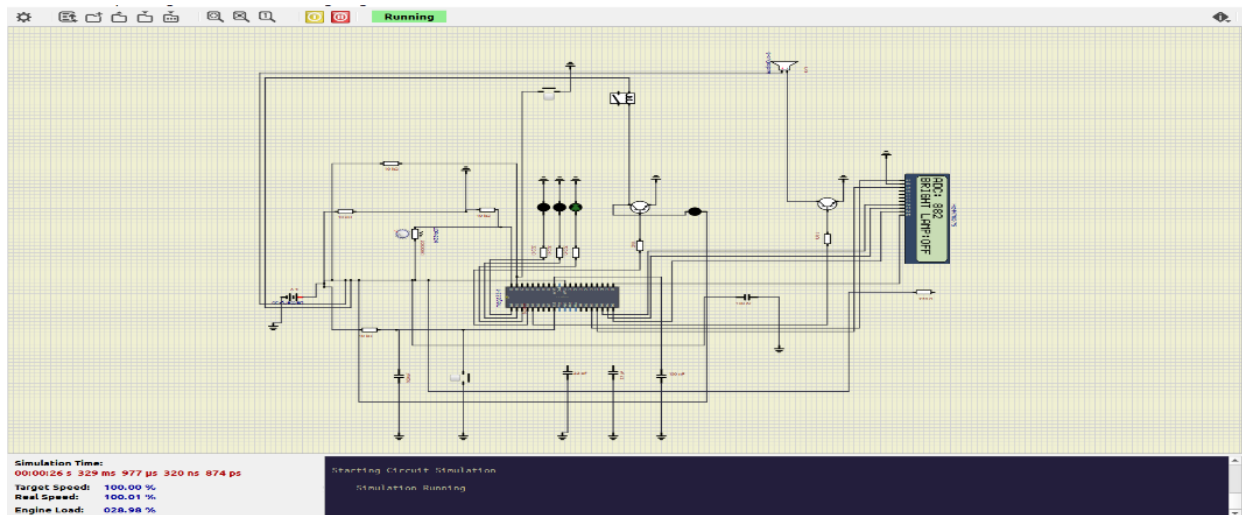


Schematic Element	Implementation
ATmega32 MCU	PA0/ADC0 and all GPIO pins shown with clear net labels
LDR Voltage Divider	R5 (LDR03) + R1 (10kohm) divider feeding PA0 with net label LDR_ADC
ADC Reference	AREF with 100nF filtering capacitor; AVCC connected to +5V supply
LED Drivers	Three LEDs (Green D5, Yellow D6, Red D4) with 220 ohm resistors on PB0-PB2
Lamp Control Driver	NPN BC547 (Q1) transistor driven by PC0 through R3 (1kohm); relay K1
Buzzer Driver	NPN BC547 (Q2) transistor driven by PC1 through R8 (1kohm)
Manual Override	SW1 on PA1 with internal pull-up enabled in firmware
LCD Connector	16x2 WC1602A in 4-bit mode on PD0-PD1 (RS/EN) and PD4-PD7 (data)
Reset Circuit	SW2 + C6 (1uF) + R11 (10kohm) pull-up on RESET pin 9
Clock Circuit	16MHz crystal Y1 with 22pF load capacitors C1 and C2
Net Labels Used	LDR_ADC, BTN_OVERRIDE, LAMP_OUT, LED_BRIGHT, LED_MEDIUM, LED_DARK, BUZZER

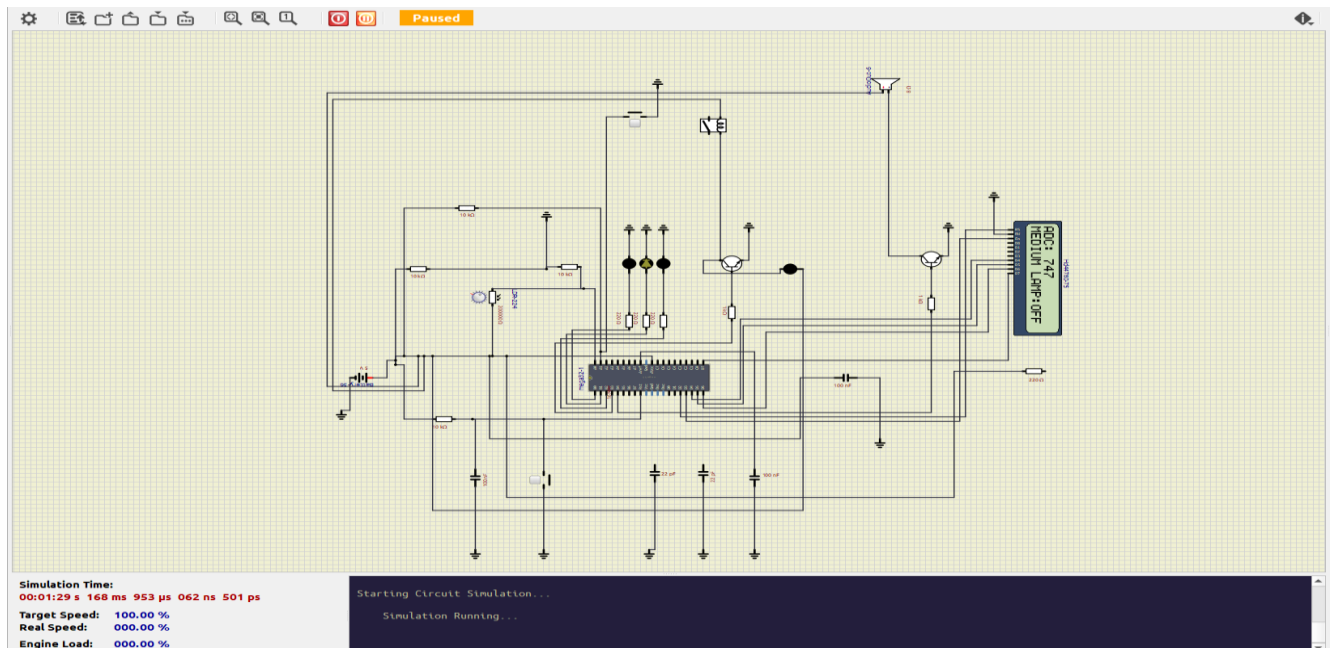
10. SimulIDE Simulation

The circuit was built and tested in SimulIDE using the ATmega32-16PU component loaded with the compiled .hex firmware. An LDR-250 component was used to simulate varying light intensity by adjusting the Lux value during simulation.

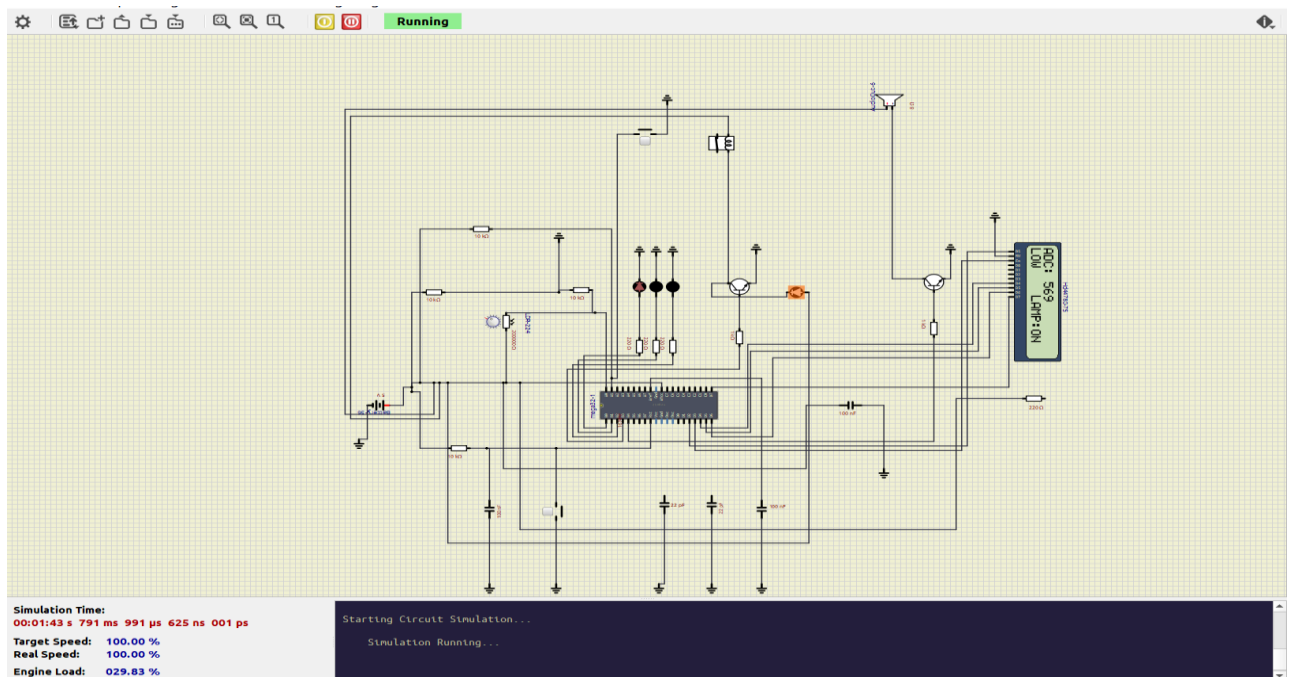
BRIGHT State (Green LED ON – Lamp OFF)



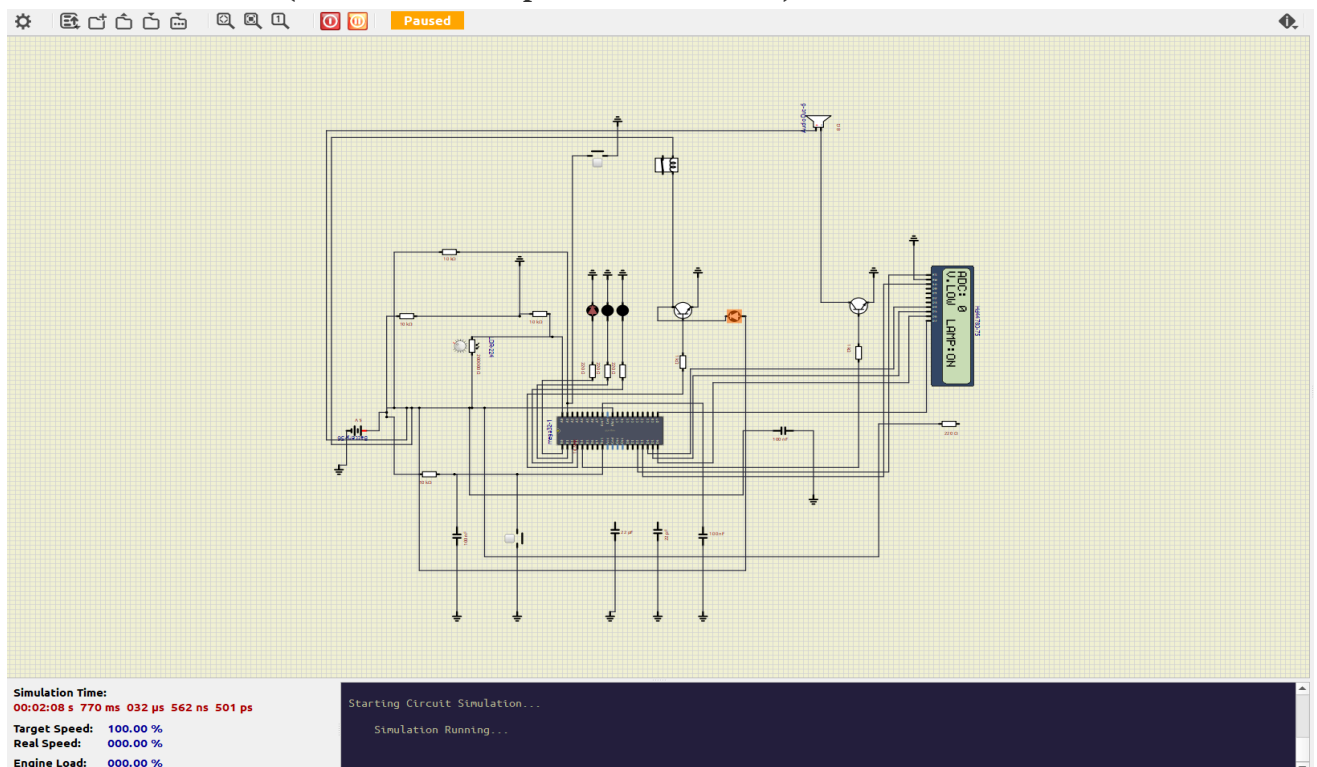
MEDIUM State (Yellow LED ON)



LOW LIGHT State (Red LED ON – Lamp ON)



VERY DARK State (Red LED + Lamp ON + Buzzer ON)



11. Firmware Overview

Firmware Task	Implementation	Register / Code
ADC Initialization	Configure reference, prescaler, channel	ADMUX=(1<<REFS0); ADCSRA=(1<<ADEN) (1<<ADPS2) (1<<ADPS1)
GPIO Configuration	Set DDR registers for all ports	DDRA=0x00; DDRB=0x07; DDRC=0x03; DDRD=0xF3
Pull-up Enable	Enable internal pull-up on PA1 button	PORTA = (1<<PA1)
LDR Reading	Read 10-bit ADC from PA0 continuously	ReadADC(0) – polls ADSC until conversion complete
Threshold Decision	4-state classification of light level	if(light>800) BRIGHT; elif(>600) MEDIUM; elif(>400) LOW; else DARK
LED Control	Turn ON correct LED for each state	PORTB = (1<<PB2) or (1<<PB1) or (1<<PB0) per state
Relay Control	Relay ON for LOW and DARK states	PORTC = (1<<PC0); cleared for BRIGHT and MEDIUM
Buzzer Control	Buzzer ON for DARK and ERROR states	PORTC = (1<<PC1); cleared otherwise
Manual Override	Toggle lamp when PA1 button pressed	manualMode ^= 1; PORTC ^= (1<<PC0); 300ms debounce
LDR Error Detection	Detect open or shorted LDR sensor	if(light<=5 light>=1018) enter ERROR state
LCD Display	Show state message and startup screen	LCD_String("BRIGHT"/"MEDIUM"/"LOW LIGHT"/"VERY DARK")

Key Firmware Files

File	Location	Description
main.c	firmware/src/	Main application – ADC init, state machine, LCD, output control
lcd.c	firmware/src/	LCD driver – 4-bit HD44780 init, command, char, and string functions
lcd.h	firmware/include/	LCD pin definitions (RS=PD0, EN=PD1, D4-D7=PD4-PD7) and prototypes
*.hex	firmware/hex/	Compiled Intel HEX file loaded into SimulIDE and physical MCU

12. Test Results

All test cases were verified in SimulIDE simulation. The table below documents expected and actual outputs for each condition.

TC#	Test Case	Input Condition	Expected Output	Result
TC-01	System Startup	Power ON	LCD: "SMART LIGHT / SYSTEM READY" for 2 seconds	PASS
TC-02	BRIGHT Condition	LDR high Lux (>800)	Green LED ON, Relay OFF, LCD: BRIGHT	PASS
TC-03	MEDIUM Condition	LDR medium Lux (>600)	Yellow LED ON, Relay OFF, LCD: MEDIUM	PASS
TC-04	LOW Condition	LDR low Lux (>400)	Red LED ON, Relay ON, LCD: LOW LIGHT	PASS
TC-05	DARK Condition	LDR very low Lux	Red LED + Relay + Buzzer ON, LCD: VERY DARK	PASS
TC-06	LDR ERROR	ADC < 5 or > 1018	Red LED + Buzzer ON, LCD: LDR ERROR	PASS
TC-07	Manual Override	Press SW1 PA1	manualMode toggles, LCD shows MANUAL MODE	PASS
TC-08	Manual Lamp Toggle	In manual mode	Relay toggles ON/OFF, LCD shows LAMP TOGGLED	PASS
TC-09	Return to Auto	Press SW1 again	System returns to AUTO MODE with ADC-based control	PASS
TC-10	Firmware Build	MPLAB X compile	Zero errors, .hex file generated successfully	PASS

All 10 test cases PASSED in SimulIDE simulation.

13. GitHub Repository

The complete project has been uploaded to a public GitHub repository with a structured folder layout and meaningful commit history as required.

Repository Name: ATmega32_LDR_Automatic_Lighting

Repository URL: https://github.com/josbertbaingana2002-bot/ATmega32_LDR_Automatic_Lighting/blob/main/KICAD1/AUTOMATIC%20LIGHT%20SENSOR-backups/AUTOMATIC%20LIGHT%20SENSOR-2026-07-11_144634.zip

Folder / File	Contents
README.md	Project overview, objectives, setup instructions
firmware/src/	main.c, lcd.c – complete source code files
firmware/include/	lcd.h – LCD pin definitions and function prototypes
firmware/hex/	ATmega32_LDR_Lighting.X.production.hex
kicad/schematic/	KiCad schematic file, project file, and PNG export
kicad/pcb/	PCB layout file (.kicad_pcb) and PNG export
kicad/gerber/	All Gerber files (.gbr) and drill files (.drl)
simulide/circuit/	AUTOMATIC LDR.sim1 – complete SimulIDE circuit file
simulide/screenshots/	Screenshots of all 6 simulation test states
documentation/block_diagram.png	System block diagram
documentation/flowchart.png	Firmware flowchart
documentation/pin_mapping_table.md	Complete ATmega32 pin mapping table
documentation/adc_threshold_table.md	ADC threshold values and light conditions
documentation/test_results.md	Simulation test case results
media/demonstration_video_link.txt	Link to project demonstration video

Commit History

1. Initial project structure created
2. Added ADC firmware for LDR reading
3. Added light threshold decision logic
4. Added automatic lamp control firmware
5. Added manual override function
6. Added KiCad schematic design
7. Added SimulIDE simulation circuit
8. Added PCB layout and Gerber files
9. Added test results and project documentation

14. Conclusion

ADC-Based Light Sensing

The ATmega32 ADC was successfully configured to read analog voltage from the LDR voltage divider on PA0, converting it to a 10-bit digital value (0–1023) using AVCC as the 5V reference.

4-State Automatic Control

The firmware correctly classifies ambient light into four states (BRIGHT, MEDIUM, LOW, DARK) using defined ADC thresholds and drives the relay, buzzer, LEDs, and LCD display accordingly.

Manual Override

The push button on PA1 reliably toggles manual mode with 300ms software debounce, allowing the user to control the lamp independently of the light sensor readings.

Error Detection

The system detects abnormal LDR readings ($\text{ADC} < 5$ or > 1018) and enters an error state with buzzer alert and LCD error message, providing robust fault indication.

Simulation Verification

All 10 test cases were verified in SimulIDE with correct behavior across all light states, manual override, and error conditions before hardware implementation.

Documentation and Repository

A complete project documentation report and structured GitHub repository has been prepared including schematic, PCB, firmware, Gerber files, diagrams, and test results with meaningful commit history.

This project demonstrates a complete embedded systems design cycle — from problem understanding through hardware design, firmware development, simulation testing, PCB layout, and documentation — fulfilling all CT 321 PBL 18 assessment requirements.